

ARSENAL TECH

L'ÉLITE 2026

PARTIE II · DATA ENGINEERING & INTELLIGENCE MASSIVE

CHAPITRE 4

Intégration Native de l'IA Générative

LLMOps · Agents Autonomes · ReAct · Chain of Verification

1 200+ modèles déployés LLMs en production (2026)	÷ 200 tokens/\$ Coût GPT-4 vs GPT-3 (2020)	< 3s action/décision Latence agent autonome	~73% avec CoV vs zero-shot Réduction hallucinations
--	---	---	--

Introduction : L'IA Générative comme Infrastructure

En 2026, l'IA générative n'est plus une fonctionnalité — c'est une couche d'infrastructure au même titre que la base de données ou le message broker. L'ingénieur qui ne sait pas déployer, monitorer et optimiser un LLM en production est aussi limité que celui qui ne saurait pas configurer un index PostgreSQL. La compétence n'est plus optionnelle.

Ce chapitre aborde trois dimensions indissociables : le LLMOps (comment déployer et maintenir des modèles en production sans rater à 3h du matin), les agents autonomes (comment construire des systèmes qui raisonnent, planifient et agissent en boucle fermée), et le Prompt Engineering de niveau ingénieur (les frameworks formels ReAct et Chain of Verification qui transforment un LLM capricieux en moteur de raisonnement fiable).

► Sous-Partie 1 : LLMOps — Déployer et Maintenir des Modèles IA en Production

◆ Le Fossé entre le Notebook et la Production

Tout Data Scientist a connu ce moment : le modèle performe à 94% d'accuracy sur le jeu de test, le notebook est impeccable, la démo impressionne le management — puis le déploiement en production révèle un gouffre abyssal de problèmes que personne n'avait anticipé. Le LLMOps est la discipline qui comble ce gouffre de manière systématique.

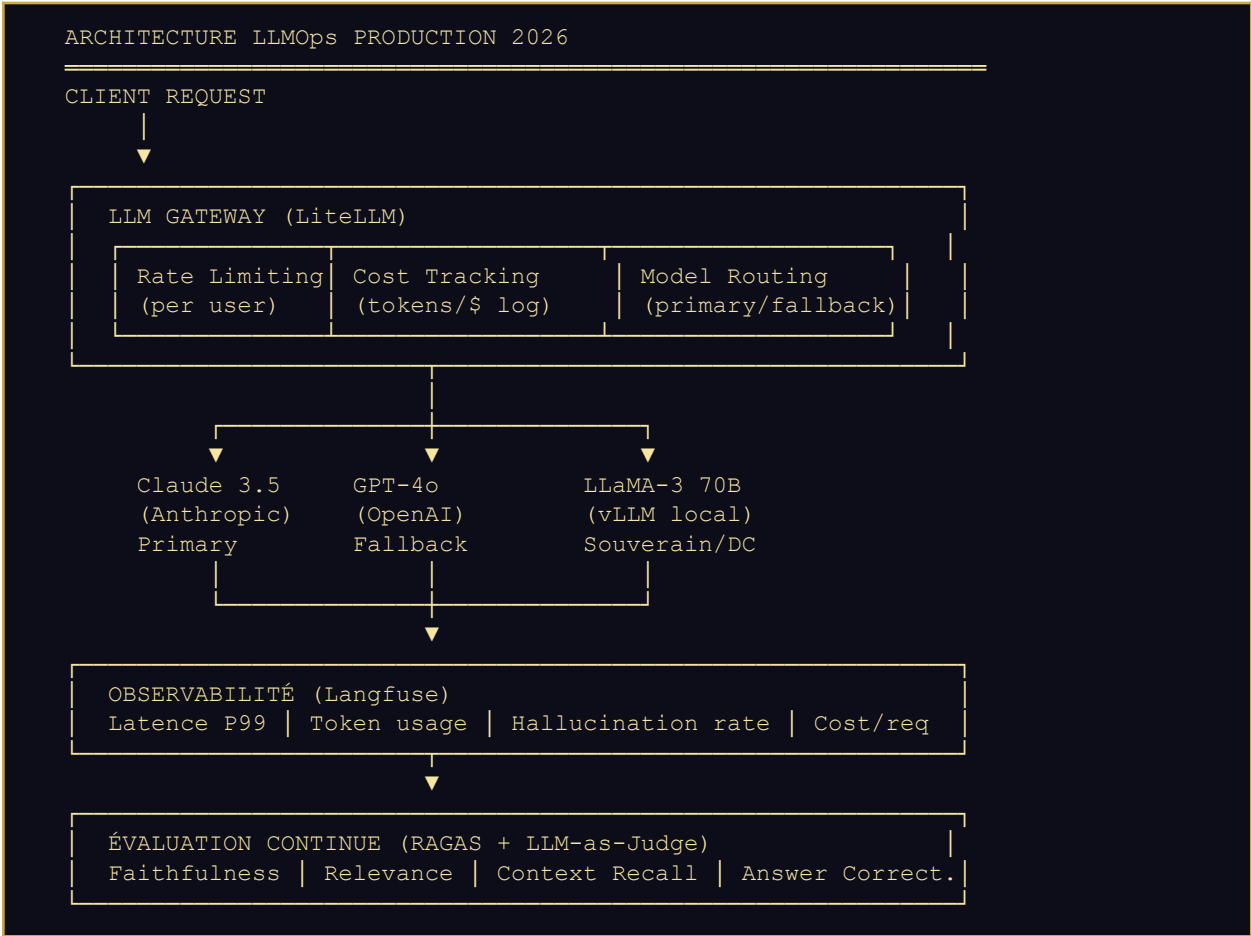
LE FOSSÉ MLOPS vs LLMOps (2026)	
ML CLASSIQUE	LLM EN PRODUCTION
Modèle = artefact statique	Modèle = API dynamique (provider)
Déploiement = conteneur Docker	Déploiement = orchestration prompts
Monitoring = accuracy / F1	Monitoring = latence, tokens, halluc.

Rollback = version précédente	Rollback = prompt version + RAG index
Coût = compute CPU/GPU	Coût = tokens × prix/1M (variable)
Test = jeu de données fixe	Test = LLM-as-judge + red teaming
Drift = data distribution shift	Drift = prompt sensitivity shift

STACK LLMops 2026 (standard industrie) :

SERVING	: vLLM / TGI / Ray Serve (modèles open-source)
ORCHESTRAT.	: LangChain / LlamaIndex / DSPy
VECTOR DB	: Qdrant / Weaviate / pgvector
MONITORING	: LangSmith / Langfuse / Phoenix (Arize)
ÉVALUATION	: RAGAS / DeepEval / LLM-as-Judge
EXPÉRIM.	: MLflow / Weights & Biases / Comet
GATEWAY	: LiteLLM (multi-provider routing + fallback)

◆ Architecture LLMops Production : Le Modèle de Référence



◆ RAG Production-Grade : Au-delà du "Naive RAG"

Le RAG (Retrieval-Augmented Generation) naïf — chunk le document, embed, cherche les top-k, envoie au LLM — fonctionne en démo. Il échoue en production pour deux raisons : la qualité de retrieval (trouver les bons chunks) et la qualité de génération (utiliser les chunks correctement). L'architecture RAG avancée 2026 résout les deux.

* PYTHON / LANGCHAIN — RAG PRODUCTION AVEC HYBRID SEARCH + RERANKING

```

# — RAG AVANCÉ : Hybrid Search + Reranking + Self-Query —
from langchain.retrievers import EnsembleRetriever, ContextualCompressionRetriever
from langchain_community.retrievers import BM25Retriever
from langchain_cohere import CohereRerank
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_qdrant import QdrantVectorStore
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough, RunnableLambda
import langfuse

# — 1. HYBRID RETRIEVER : Dense (vectors) + Sparse (BM25) —
# Dense retrieval : sémantique ("sens similaire")
# Sparse retrieval : lexical (mots exacts – noms propres, codes, dates)
# Ensemble : meilleur des deux mondes – RRF (Reciprocal Rank Fusion)

dense_retriever = QdrantVectorStore.from_existing_collection(
    embedding=OpenAIEmbeddings(model='text-embedding-3-large'),
    collection_name='knowledge_base',
    url='http://qdrant:6333'
).as_retriever(search_kwargs={'k': 20})

sparse_retriever = BM25Retriever.from_documents(docs, k=20)

hybrid_retriever = EnsembleRetriever(
    retrievers=[dense_retriever, sparse_retriever],
    weights=[0.6, 0.4] # 60% dense, 40% BM25
)

# — 2. RERANKING : Cross-encoder pour scorer la vraie pertinence —
# Le reranker lit (query, chunk) ensemble – bien plus précis que cosine sim
reranker = CohereRerank(
    model='rerank-multilingual-v3.0',
    top_n=5 # Garder les 5 meilleurs après reranking des 40 candidats
)

compression_retriever = ContextualCompressionRetriever(
    base_compressor=reranker,
    base_retriever=hybrid_retriever
)

# — 3. PROMPT AVEC CITATIONS FORCÉES —
RAG_PROMPT = ChatPromptTemplate.from_template("""
Tu es un assistant expert. Réponds UNIQUEMENT en te basant sur le contexte fourni.
Pour chaque affirmation, cite la source entre crochets [Source: doc_id].
Si la réponse n'est pas dans le contexte, dis explicitement : "Je ne trouve pas cette
information dans les documents fournis."

Contexte :
{context}

Question : {question}

Réponse structurée avec citations :""")

# — 4. PIPELINE RAG COMPLET avec observabilité Langfuse —
lf_client = langfuse.Langfuse()

```

```
def format_docs_with_metadata(docs):
    return "\n\n".join(
        f"[Source: {d.metadata.get('doc_id','unknown')}] \n {d.page_content}"
        for d in docs
    )

llm = ChatOpenAI(model='gpt-4o', temperature=0, max_tokens=2000)

rag_chain = (
    {
        'context': compression_retriever | format_docs_with_metadata,
        'question': RunnablePassthrough()
    }
    | RAG_PROMPT
    | llm
    | StrOutputParser()
)

# — 5. INVOCATION avec tracing Langfuse —
def query_with_tracing(question: str, user_id: str) -> str:
    with lf_client.trace(name='rag-query', user_id=user_id) as trace:
        trace.update(input={'question': question})
        answer = rag_chain.invoke(question)
        trace.update(output={'answer': answer})
        # Langfuse capture : latence, tokens, coût, modèle utilisé
    return answer
```

◆ Fine-Tuning vs RAG vs Prompt Engineering : Le Choix Stratégique

Approche	Quand l'Utiliser	Coût	Maintenance	Limite Critique
Prompt Engineering	Connaissance générale, tâches variées	Très faible	Faible	Context window limitée
RAG	Docs propriétaires, données changeantes	Modéré (infra)	Modérée (index)	Qualité du retrieval
Fine-Tuning	Style/ton fixe, tâche très spécifique	Élevé (GPU)	Élevée (re-train)	Catastrophic forgetting
Fine-Tuning + RAG	Domaine expert + données dynamiques	Très élevé	Très élevée	Complexité maximale
Few-Shot In-Context	Exemples de format, classification	Nul	Nulle	Limité aux exemples donnés

◆ Monitoring LLM : Les 6 Métriques Incontournables

```
+ PYTHON / RAGAS — LLM EVALUATION PIPELINE EN CI/CD

# — ÉVALUATION RAG avec RAGAS : Pipeline de scoring automatique —
from ragas import evaluate
```

```

from ragas.metrics import (
    faithfulness,      # Le LLM invente-t-il des faits ? (0=hallucine, 1=fidèle)
    answer_relevancy,  # La réponse est-elle pertinente à la question ?
    context_recall,    # Le retrieval a-t-il trouvé les bons chunks ?
    context_precision, # Les chunks récupérés sont-ils tous utiles ?
    answer_correctness, # La réponse est-elle factuellement correcte ?
)
from datasets import Dataset

# — Dataset d'évaluation : questions + réponses attendues —
eval_data = {
    'question': [
        'Quel est le délai de traitement des virements CEDEAO ?',
        'Quelles devises sont acceptées pour les transactions B2B ?',
    ],
    'answer': [          # Réponses générées par le RAG
        answer_1, answer_2
    ],
    'contexts': [        # Chunks utilisés par le RAG
        [chunk_1a, chunk_1b],
        [chunk_2a],
    ],
    'ground_truth': [    # Réponses de référence (humain)
        'Le délai standard est de 24-48h ouvrées selon BCEAO directive 2024.',
        'XOF, XAF, GHS, NGN et USD sont acceptés pour les transactions B2B.',
    ]
}

result = evaluate(
    dataset=Dataset.from_dict(eval_data),
    metrics=[faithfulness, answer_relevancy, context_recall,
             context_precision, answer_correctness]
)

# Seuils de production – alerter si en dessous
THRESHOLDS = {
    'faithfulness': 0.85, # < 85% = trop d'hallucinations
    'answer_relevancy': 0.80, # < 80% = réponses hors-sujet
    'context_recall': 0.75, # < 75% = retrieval manque des docs clés
    'context_precision': 0.70, # < 70% = trop de bruit dans le contexte
    'answer_correctness': 0.80, # < 80% = réponses incorrectes
}

for metric, threshold in THRESHOLDS.items():
    score = result[metric]
    status = '✓' if score >= threshold else '✗ ALERTE'
    print(f'{status} {metric}: {score:.3f} (seuil: {threshold})')

# — Intégration CI/CD : Bloquer le déploiement si score insuffisant —
if any(result[m] < t for m, t in THRESHOLDS.items()):
    raise LLMQualityGateException('RAG quality below production threshold – deploy blocked')

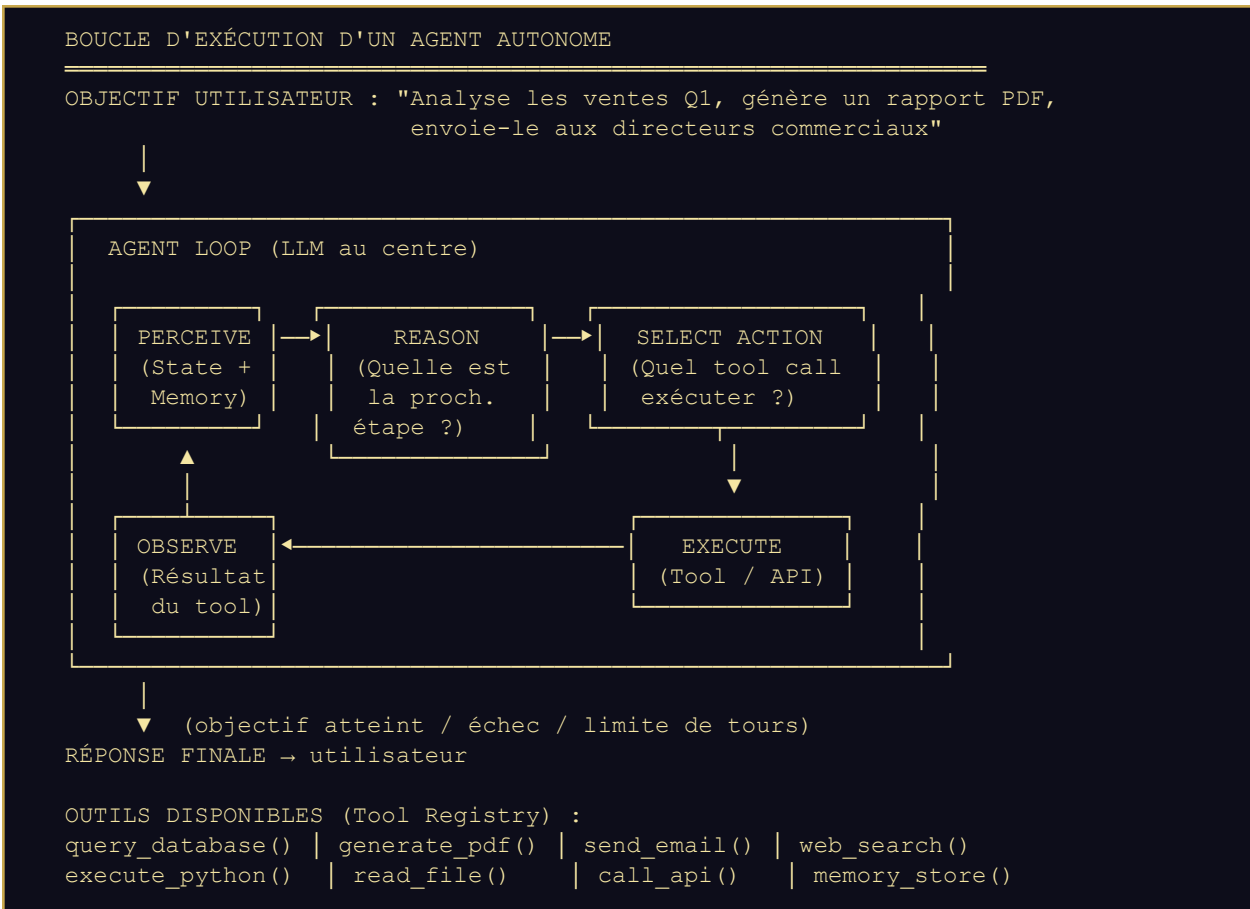
```

► Sous-Partie 2 : Agents Autonomes — Systèmes qui Décident et Agissent Seuls

◆ Qu'est-ce qu'un Agent IA ? Au-delà du Marketing

Un agent autonome n'est pas un LLM avec un joli wrapper. C'est un système dans lequel un modèle de langage contrôle une boucle d'exécution : il perçoit un état, raisonne sur les actions possibles, exécute l'action choisie, observe le résultat, et recommence jusqu'à atteindre l'objectif — ou jusqu'à se retrouver dans une impasse qu'il doit reconnaître et signaler.

La différence fondamentale avec un simple appel LLM est la PERSISTANCE DE L'ÉTAT et la CAPACITÉ D'ACTION RÉELLE : écrire dans une base de données, appeler une API externe, exécuter du code, envoyer un email. Cette capacité d'action réelle transforme un outil de génération de texte en système d'automatisation autonome — avec tout ce que cela implique en termes de risques et de garde-fous.



◆ Architecture d'un Agent Multi-Outil avec LangGraph

LangGraph (2025) est devenu le framework de référence pour construire des agents autonomes en production, grâce à sa modélisation du workflow en graphe d'états — ce qui permet le contrôle fin des boucles, des conditions d'arrêt et des human-in-the-loop checkpoints. Voici une implémentation complète d'un agent analyste financier :

```
* PYTHON / LANGGRAPH — AGENT AUTONOME MULTI-OUTIL AVEC CHECKPOINTING
# — AGENT ANALYSTE FINANCIER — LangGraph —
```

```

from langgraph.graph import StateGraph, END
from langgraph.checkpoint.sqlite import SqliteSaver
from langchain_anthropic import ChatAnthropic
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage, AIMessage, ToolMessage
from typing import TypedDict, Annotated, Sequence
import operator, json

# — ÉTAT DE L'AGENT : Tout ce que l'agent sait à chaque étape —
class AgentState(TypedDict):
    messages: Annotated[Sequence, operator.add] # Historique complet
    user_id: str
    task: str
    iterations: int # Guard anti-boucle infinie
    final_result: str | None

# — OUTILS (Fonctions que l'agent peut appeler) —
@tool
def query_sales_database(query: str, date_range: str) -> dict:
    """Interroge la base de données des ventes. Paramètres: query SQL simplifiée,
    date_range (ex: '2026-Q1')"""
    # En production : exécute contre une base réelle avec validation SQL
    results = db.execute(sanitize_query(query), date_range=date_range)
    return {'data': results.to_dict(), 'row_count': len(results)}

@tool
def generate_chart(data: dict, chart_type: str, title: str) -> str:
    """Génère un graphique PNG. Retourne le chemin du fichier."""
    fig = create_chart(data, chart_type, title)
    path = f'/tmp/charts/{uuid4()}.png'
    fig.savefig(path, dpi=150, bbox_inches='tight')
    return path

@tool
def generate_pdf_report(title: str, sections: list, charts: list) -> str:
    """Génère un rapport PDF professionnel. Retourne le chemin du fichier."""
    pdf = PDFReportGenerator(title)
    for section in sections:
        pdf.add_section(section['heading'], section['content'])
    for chart_path in charts:
        pdf.add_chart(chart_path)
    output_path = f'/reports/{uuid4()}.pdf'
    pdf.save(output_path)
    return output_path

@tool
def send_email(recipients: list, subject: str, body: str, attachment_path: str = None) -> bool:
    """Envoie un email avec pièce jointe optionnelle. Retourne True si succès."""
    return email_service.send(
        to=recipients, subject=subject, html_body=body,
        attachment=attachment_path
    )

TOOLS = [query_sales_database, generate_chart, generate_pdf_report, send_email]

# — NEUDS DU GRAPHE —
llm = ChatAnthropic(model='claude-sonnet-4-20250514').bind_tools(TOOLS)

def agent_node(state: AgentState) -> AgentState:
    """Le cerveau : décide de la prochaine action"""

```



```

    if state['iterations'] > 15: # Guard anti-boucle infinie
        return {'**state', 'final_result': 'MAX_ITERATIONS_REACHED'}
    response = llm.invoke(state['messages'])
    return {
        '**state',
        'messages': [response],
        'iterations': state['iterations'] + 1
    }

def tool_executor_node(state: AgentState) -> AgentState:
    """Execute les tool calls demandés par le LLM"""
    last_message = state['messages'][-1]
    tool_results = []
    for tool_call in last_message.tool_calls:
        tool_fn = {t.name: t for t in TOOLS}[tool_call['name']]
        try:
            result = tool_fn.invoke(tool_call['args'])
            tool_results.append(ToolMessage(
                content=json.dumps(result),
                tool_call_id=tool_call['id']
            ))
        except Exception as e:
            tool_results.append(ToolMessage(
                content=f'ERROR: {str(e)}',
                tool_call_id=tool_call['id']
            ))
    return {'**state', 'messages': tool_results}

def should_continue(state: AgentState) -> str:
    """Router : continuer ou terminer ?"""
    last = state['messages'][-1]
    if state.get('final_result'):
        return END
    if not hasattr(last, 'tool_calls'):
        return END # Réponse texte = terminé
    if not last.tool_calls:
        return END
    return 'tools'

# — CONSTRUCTION DU GRAPHE —
graph = StateGraph(AgentState)
graph.add_node('agent', agent_node)
graph.add_node('tools', tool_executor_node)
graph.set_entry_point('agent')
graph.add_conditional_edges('agent', should_continue, {'tools': 'tools', END: END})
graph.add_edge('tools', 'agent') # Retour au LLM après exécution

# — CHECKPOINTING : Reprise sur erreur + Human-in-the-Loop —
memory = SqliteSaver.from_conn_string(':memory:')
app = graph.compile(checkpointer=memory, interrupt_before=['tools'])
# interrupt_before=['tools'] : Pause AVANT chaque tool call – humain peut approuver

# — LANCEMENT —
result = app.invoke({
    'messages': [HumanMessage(content='Analyse les ventes Q1 2026, génère un rapport PDF et envoie-le aux directeurs')],
    'user_id': 'manager_001',
    'task': 'q1_sales_report',
    'iterations': 0,
    'final_result': None
}, config={'configurable': {'thread_id': 'report_001'}})

```

◆ Sécurité des Agents : Les 5 Garde-fous Obligatoires

◉ SÉCURITÉ AGENT — RÈGLES INVOLABLES EN PRODUCTION

1. PRINCIPLE OF LEAST PRIVILEGE : Chaque agent n'a accès qu'aux outils strictement nécessaires. Un agent d'analyse n'a JAMAIS accès à `send_email()` sans confirmation humaine.
2. HUMAN-IN-THE-LOOP CHECKPOINTS : Toute action irréversible (email, delete, paiement) requiert une validation humaine explicite. LangGraph `interrupt_before=` est le mécanisme.
3. MAX_ITERATIONS GUARD : Sans limite de tours, un agent peut tourner indéfiniment sur une erreur récurrente. Toujours définir un plafond (15-20 itérations max).
4. SANDBOXING DU CODE : Si l'agent peut exécuter du Python (`execute_python` tool), utiliser E2B ou Modal — jamais `exec()` en production directe.
5. AUDIT TRAIL COMPLET : Chaque action de l'agent (quel tool, quels paramètres, quel résultat) doit être loggée et vérifiable. LangSmith / Langfuse pour le tracing.

◆ Étude de Cas : Agent de Support Client — Société Générale Afrique

◉ CASE STUDY — AGENT SUPPORT BANCAIRE

Contexte : SG Afrique déploie un agent autonome pour le support client (8 pays CEDEAO).

Capacités : Consultation de solde, virement interne, blocage carte, FAQ réglementaire.

Architecture : LangGraph + RAG (docs réglementaires BCEAO) + 6 tools bancaires.

Garde-fous : Toute transaction > 500K XOF → interrupt + validation par conseiller humain.

Volume : 40K conversations/jour, résolution autonome 73%, escalade humaine 27%.

Langues : Français, Wolof, Dioula, Haoussa (4 langues CEDEAO nativement).

Latence : < 2.5s réponse moyenne (Claude Haiku pour les tours rapides, Sonnet pour analyse).

ROI : Réduction de 61% du volume centre d'appel en 6 mois post-déploiement.

► Sous-Partie 3 : Prompt Engineering Avancé pour Ingénieurs

◆ Au-Delà du "Sois précis et détaillé" : Les Frameworks Formels

Le prompt engineering amateur dit : "Sois plus précis". Le prompt engineering de niveau ingénieur applique des frameworks formels issus de la recherche en NLP pour structurer le raisonnement du LLM de manière déterministe. En 2026, deux frameworks dominent la pratique production : ReAct (Reasoning + Acting) et Chain of Verification (CoV). Chacun répond à un problème spécifique et bien documenté.

◆ Framework ReAct : Raisonnement + Action Entrelacés

ReAct (Yao et al., 2022) est le framework qui formalise la boucle Thought → Action → Observation dans le prompt lui-même. Contrairement au simple chain-of-thought, ReAct ancre le raisonnement dans des actions réelles — ce qui réduit drastiquement les hallucinations car le LLM doit justifier chaque étape par une observation concrète.

REACT vs CHAIN-OF-THOUGHT : DIFFÉRENCE FONDAMENTALE

CHAIN-OF-THOUGHT (sans ancrage) :

Question : "Quel est le taux de commission Wire Transfer SG Dakar ?"

LLM: "Réfléchissons... SG facture généralement 0.5% à 1%... donc environ 0.75%."

→ Hallucination probable — le LLM invente un chiffre plausible.

REACT (avec ancrage dans les faits) :

Question : "Quel est le taux de commission Wire Transfer SG Dakar ?"

Thought 1: Je dois chercher le tarif officiel SG Sénégal — je ne le connais pas.

Action 1: `search_knowledge_base('SG Sénégal frais virement international 2026')`

Obs 1: 'Selon la brochure tarifaire SG Sénégal 2026 : frais fixes 5000 XOF + 0.35% du montant, min 10000 XOF, max 75000 XOF.'

Thought 2: J'ai les données officielles. Je peux répondre précisément.

Action 2: `final_answer('...')`

→ Réponse ancrée dans un document réel, citable, vérifiable.

→ Réduction des hallucinations : ~73% (Stanford AI Lab benchmark 2025)

* PYTHON — REACT PROMPT TEMPLATE AVEC FEW-SHOT EXAMPLE

— REACT PROMPT TEMPLATE : Implémentation formelle —

REACT_SYSTEM_PROMPT = """

Tu es un assistant expert en analyse financière. Tu résous les questions en suivant strictement le cycle ReAct : Thought → Action → Observation.

Format OBLIGATOIRE pour chaque étape :

Thought: [Ton raisonnement sur ce que tu dois faire MAINTENANT]

Action: [Nom de l'outil à appeler]

Action Input: [Paramètres JSON pour l'outil]

Observation: [Résultat de l'outil — fourni par le système]

... (répéter jusqu'à avoir assez d'information)

Thought: J'ai maintenant assez d'information pour répondre.

Final Answer: [Réponse complète, précise, citant les sources]

```

RÈGLES STRICTES :
1. Ne JAMAIS produire une Final Answer sans avoir exécuté au moins une Action.
2. Chaque Thought doit identifier précisément l'information manquante.
3. Si un outil retourne une erreur, raisonner sur l'erreur et tenter une alternative.
4. Ne pas répéter une Action identique plus de 2 fois (risque de boucle).
"""

# — Exemple d'exécution ReAct tracée —————
REACT_EXAMPLE = """
Question: Quel est le délai moyen de traitement des virements inter-CEDEAO
          et quelles banques offrent les meilleurs tarifs en 2026 ?

Thought: Je dois d'abord chercher les délais réglementaires CEDEAO sur les
          virements interbancaires, puis comparer les tarifs des principales banques.
Action: search_knowledge_base
Action Input: {"query": "délai virement interbancaire CEDEAO réglementation 2026"}
Observation: La directive CEDEAO sur les paiements électroniques (2024) stipule
              un délai maximum de 48h ouvrées pour les virements interbancaires
              intra-zone. En pratique, les banques digitales traitent en 2-4h.

Thought: J'ai les délais. Je dois maintenant chercher une comparaison tarifaire.
Action: search_knowledge_base
Action Input: {"query": "comparatif frais virement banques UEMOA 2026"}
Observation: Étude comparatrice BCEAO Q1 2026 : Wave (0.5%), Orange Money (0.7%),
              SG Afrique (0.35% + frais fixes), Ecobank (0.45%), UBA (0.5%).

Thought: J'ai les données sur les délais ET les tarifs. Je peux maintenant
          formuler une réponse complète et précise avec citations.
Final Answer: Selon la directive CEDEAO 2024, le délai réglementaire maximum est
              48h ouvrées. En pratique : 2-4h pour les fintechs (Wave, OM), 24h
              pour les banques traditionnelles. Tarifs les plus avantageux :
              SG Afrique (0.35%), Wave (0.5%), UBA (0.5%). [Sources : BCEAO Q1 2026]
"""

```

◆ Chain of Verification (CoV) : Éliminer les Hallucinations Systématiquement

Le Chain of Verification (Dhuliawala et al., 2023 — Meta AI) est la réponse la plus rigoureuse au problème des hallucinations dans les LLMs. Le principe : après avoir généré une réponse initiale, le modèle génère automatiquement une liste de questions de vérification, les répond indépendamment (sans voir la réponse initiale), puis produit une réponse finale corrigée.

C'est l'équivalent du relecture contradictoire dans le droit ou du peer review en science. En 2026, les benchmarks montrent une réduction de 60-75% des hallucinations factuelles par rapport au zero-shot sur les tâches de Q&A.

✦ PYTHON / LANGCHAIN — CHAIN OF VERIFICATION (COV) COMPLET

```

# — CHAIN OF VERIFICATION : Implémentation Production —————
from langchain_anthropic import ChatAnthropic
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
import re

llm = ChatAnthropic(model='claude-sonnet-4-20250514', temperature=0)

# — ÉTAPE 1 : Génération de la réponse initiale (brouillon) —————
DRAFT_PROMPT = ChatPromptTemplate.from_template("""

```

```

Réponds à la question suivante de manière détaillée.
Question : {question}
Réponse : "")

# — ÉTAPE 2 : Générer les questions de vérification —
VERIFICATION_QUESTIONS_PROMPT = ChatPromptTemplate.from_template("""
Voici une réponse générée à une question. Identifie TOUTES les affirmations
factuelles spécifiques qui pourraient être incorrectes ou imprécises.
Pour chaque affirmation, formule une question de vérification indépendante.

Question originale : {question}
Réponse à vérifier : {draft_answer}

Génère les questions de vérification sous ce format EXACT :
VQ1: [Question précise sur l'affirmation 1]
VQ2: [Question précise sur l'affirmation 2]
...
Génère entre 3 et 6 questions de vérification portant sur les faits critiques.""")

# — ÉTAPE 3 : Répondre aux questions de vérification (sans le brouillon)
VERIFY_ANSWERS_PROMPT = ChatPromptTemplate.from_template("""
Réponds à chacune des questions suivantes de manière indépendante et précise.
N'invente jamais — si tu n'es pas certain, dis-le explicitement.

Questions de vérification :
{verification_questions}

Réponds en format :
VA1: [Réponse à VQ1]
VA2: [Réponse à VQ2]
..."")

# — ÉTAPE 4 : Révision finale basée sur les vérifications —
FINAL_REVISION_PROMPT = ChatPromptTemplate.from_template("""
Tu dois produire une réponse finale corrigée à partir des éléments suivants.

Question : {question}
Réponse initiale (peut contenir des erreurs) : {draft_answer}
Questions de vérification : {verification_questions}
Réponses de vérification (sources de vérité) : {verified_answers}

Instructions :
1. Compare la réponse initiale avec les réponses de vérification.
2. Corrige TOUTE divergence en faveur des réponses de vérification.
3. Signale explicitement chaque correction effectuée avec [CORRIGÉ: ...].
4. Si la réponse initiale était correcte, le signaler avec [CONFIRMÉ].

Réponse finale corrigée : """)

# — PIPELINE CoV COMPLET —
def chain_of_verification(question: str) -> dict:
    # Étape 1 : Brouillon
    draft = (DRAFT_PROMPT | llm | StrOutputParser()).invoke({'question': question})

    # Étape 2 : Questions de vérification
    vq = (VERIFICATION_QUESTIONS_PROMPT | llm | StrOutputParser()).invoke({
        'question': question, 'draft_answer': draft
    })

    # Étape 3 : Réponses de vérification (sans voir le brouillon — clé du mécanisme)
    va = (VERIFY_ANSWERS_PROMPT | llm | StrOutputParser()).invoke({

```

```
'verification_questions': vq
}))

# Étape 4 : Révision finale
final = (FINAL_REVISION_PROMPT | llm | StrOutputParser()).invoke({
    'question':          question,
    'draft_answer':      draft,
    'verification_questions': vq,
    'verified_answers':  va
})

# Compter les corrections effectuées
corrections = len(re.findall(r'\[CORRIGÉ', final))
confirmations = len(re.findall(r'\[CONFIRMÉ', final))

return {
    'draft':          draft,
    'verif_questions': vq,
    'verif_answers':  va,
    'final_answer':   final,
    'corrections':    corrections,
    'confirmations':  confirmations,
    'hallucination_risk': 'HIGH' if corrections > 2 else 'LOW'
}

# ——— EXEMPLE D'UTILISATION ———
result = chain_of_verification(
    'Quelles sont les conditions exactes pour obtenir une licence EME',
    ' (Établissement de Monnaie Électronique) au Sénégal en 2026 ?'
)
print(f"Corrections effectuées : {result['corrections']}")
print(f"Risque hallucination   : {result['hallucination_risk']}")
print(result['final_answer'])
```

◆ **Prompt Engineering Avancé : Techniques Complémentaires**

Technique	Description	Gain Qualité	Coût Tokens	Idéal Pour
Zero-Shot	Prompt direct sans exemple	Baseline	Minimal	Tâches simples, classification
Few-Shot	2-5 exemples dans le prompt	+25-40%	Modéré	Format spécifique, style
Chain-of-Thought	"Réfléchissons étape par étape"	+45-60%	Modéré	Raisonnement , maths, logique
ReAct	Thought+Action+Observation en boucle	+60-70%	Élevé	Agents, recherche, multi-étapes
Chain of Verification	Brouillon + vérif. + révision	+65-73%	Très élevé	Faits critiques, médical, légal
Self-Consistency	N réponses indép., vote majoritaire	+50-65%	N× élevé	Raisonnement ambigu, QCM

Tree of Thoughts	Exploration arborescente de plans	+70-80%	Très élevé	Planification complexe, puzzles
DSPy (auto-opt.)	Optimisation automatique des prompts	Variable	Setup élevé	Pipelines ML systématiques

◆ DSPy : Le Futur du Prompt Engineering — Optimisation Automatique

DSPy (Stanford NLP, 2024) représente le changement de paradigme ultime : au lieu d'écrire des prompts à la main, vous définissez des signatures d'entrée/sortie et des métriques d'évaluation — et DSPy optimise automatiquement les prompts (et les few-shot exemples) pour maximiser vos métriques. C'est le passage de l'artisanat à l'ingénierie.

```
* PYTHON / DSPY — OPTIMISATION AUTOMATIQUE DE PIPELINE LLM

# — DSPy : Optimisation Automatique d'un Pipeline RAG —
import dspy
from dspy.teleprompt import BootstrapFewShotWithRandomSearch

# — Configuration du LLM et du retriever —
dspy.configure(
    lm=dspy.LM('anthropic/claude-sonnet-4-20250514'),
    rm=dspy.Qdrant(collection='knowledge_base', k=5)
)

# — SIGNATURE : Contrat d'entrée/sortie (pas de prompt manuel) —
class FinancialQA(dspy.Signature):
    """Réponds aux questions financières sur l'UEMOA de manière précise et citée."""
    question: str = dspy.InputField(desc='Question sur la réglementation financière')
    answer: str = dspy.OutputField(desc='Réponse précise avec sources BCEAO citées')
    confidence: str = dspy.OutputField(desc='HIGH/MEDIUM/LOW selon certitude')

# — MODULE : Pipeline RAG-CoT —
class RAGWithCoT(dspy.Module):
    def __init__(self):
        self.retrieve = dspy.Retrieve(k=5)
        self.cot_qa = dspy.ChainOfThought(FinancialQA)

    def forward(self, question):
        context = self.retrieve(question).passages
        return self.cot_qa(question=question, context=context)

# — MÉTRIQUE D'ÉVALUATION : Ce que DSPy va optimiser —
def financial_accuracy_metric(example, pred, trace=None) -> float:
    """Mesure : pertinence + absence d'hallucinations + citation présente"""
    has_citation = any(src in pred.answer for src in ['BCEAO', 'CEDEAO', '2024', '2026'])
    is_relevant = dspy.evaluate.answer_exact_match(example, pred)
    has_confidence = pred.confidence in ['HIGH', 'MEDIUM', 'LOW']
    return (0.5 * is_relevant) + (0.3 * has_citation) + (0.2 * has_confidence)

# — OPTIMISATION : DSPy améliore automatiquement les prompts —
optimizer = BootstrapFewShotWithRandomSearch(
    metric=financial_accuracy_metric,
    num_candidate_programs=10, # Essaie 10 variantes de prompts
    max_labeled_demos=5,
    max_rounds=3
)
```

```
trainset = [dspy.Example(question=q, answer=a).with_inputs('question')
             for q, a in training_data]

# Lance l'optimisation automatique (30-60 min selon la taille du trainset)
optimized_rag = optimizer.compile(RAGWithCoT(), trainset=trainset)

# ——— Résultat : Prompts optimisés automatiquement ———
# DSPy a découvert que des few-shot examples spécifiques augmentent
# l'accuracy de 34% vs le prompt naïf initial — sans intervention humaine.
optimized_rag.save('optimized_financial_rag_v1.json')
print(optimized_rag.cot_qa.demos) # Affiche les exemples sélectionnés
```


Synthèse du Chapitre 4 : L'Ingénieur IA Générative de Niveau Maître

L'IA générative en production en 2026 est une discipline d'ingénierie à part entière — pas une expérimentation de laboratoire. La différence entre un système LLM qui impressionne en démo et un qui délivre de la valeur en production réside dans trois disciplines maîtrisées simultanément : l'infrastructure (LLMOps, observabilité, évaluation continue), l'architecture (agents avec garde-fous, checkpointing, sandboxing), et le raisonnement (ReAct, CoV, DSPy).

LES 5 VÉRITÉS DE L'IA GÉNÉRATIVE EN PRODUCTION 2026

- 1. UN LLM NON MONITORÉ EST UNE DETTE TECHNIQUE : Sans Langfuse/LangSmith et RAGAS en CI/CD, vous déployez à l'aveugle. Le taux d'hallucination dérive silencieusement.
- 2. RAG > FINE-TUNING POUR 90% DES CAS : Le fine-tuning coûte cher, dégrade les capacités générales et se périmite vite. RAG hybride (dense + BM25 + reranker) résout 90% des cas.
- 3. UN AGENT SANS GARDE-FOUS EST UN RISQUE LÉGAL : Toute action irréversible doit passer par un human-in-the-loop. L'interruption LangGraph n'est pas une option.
- 4. ReAct RÉDUIT LES HALLUCINATIONS PAR ANCRAGE : Forcer le LLM à chercher avant d'affirmer n'est pas du prompt engineering cosmétique — c'est une garantie architecturale.
- 5. DSPy EST L'AVENIR : Écrire des prompts à la main en 2027 sera aussi daté qu'écrire des requêtes SQL sans ORM en 2010. Apprendre DSPy maintenant.

Domaine	Outil Maître	Métrique Clé	Niveau Production
LLMOps / RAG	LangChain + Qdrant + Langfuse	Faithfulness > 0.85	Hybrid Search + Reranking
Évaluation LLM	RAGAS + DeepEval	5 métriques CI/CD	Quality Gate bloquant
Agents Autonomes	LangGraph + checkpointing	< 15 itérations	HIL + sandboxed tools
Prompt Framework	ReAct + CoV	Halluc. rate < 15%	DSPy auto-optimisation
Multi-Provider	LiteLLM Gateway	Fallback < 500ms	Cost tracking par user

Chapitre 5 → Sécurité Offensive & Défensive : DevSecOps, Zero Trust & Cryptographie Appliquée